# TASK 3: WEB APPLICATION SECURITY TESTING REPORT

**Intern:** Chennari Mohammed Idris

**Date:** 19-03-2026

**Target Application:** DVWA (Damn Vulnerable Web App)

**Environment:** Metasploitable 2 (Host-Only Network)

---

## 1. Executive Summary

The objective of this task was to identify, exploit, and provide remediation strategies for common web vulnerabilities as defined by the OWASP Top 10. Using a combination of manual exploitation and proxy interception, several critical flaws were identified that could lead to full database compromise and session hijacking.

---

## 2. SQL Injection (A03:2021 – Injection)

### Vulnerability Description

The application fails to sanitize user input in the "User ID" field, allowing an attacker to manipulate the backend SQL query.

### Exploitation Steps

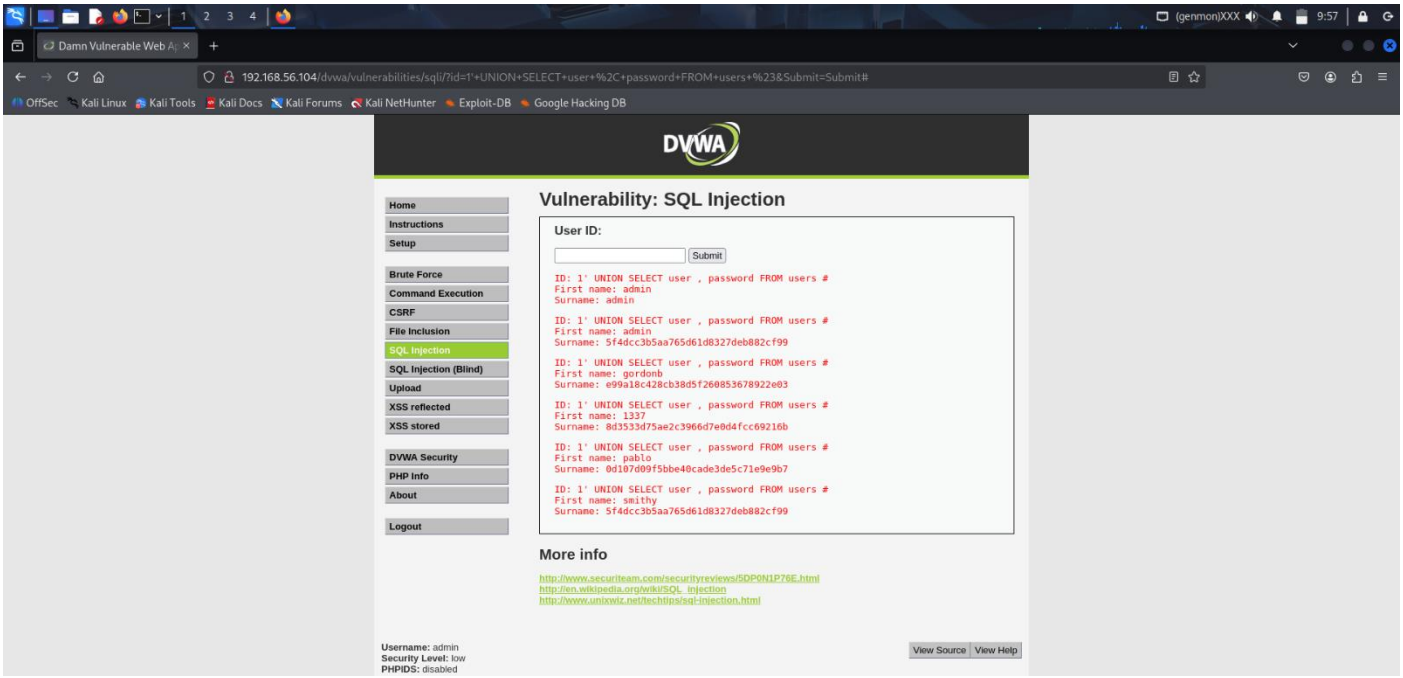
1. Navigated to the SQL Injection module.
2. Entered the payload: 1' UNION SELECT user, password FROM users #
3. The application executed the injected command and returned the entire user database.

### Risk Impact

Critical. An attacker can extract sensitive credentials, personally identifiable information (PII), and administrative hashes from the database.

### Remediation (The Fix)

Developers should implement Prepared Statements (Parameterized Queries). This ensures that the database treats user input as data, not as executable code.




## Secure Code Example (PHP):

## PHP

```
$id = $_GET['id'];

$stmt = $pdo->prepare('SELECT first_name, last_name FROM users WHERE user_id = ?');

$stmt->execute([$id]);
```

---

## 3. Cross-Site Scripting (A03:2021 – Injection/XSS)

## Vulnerability Description

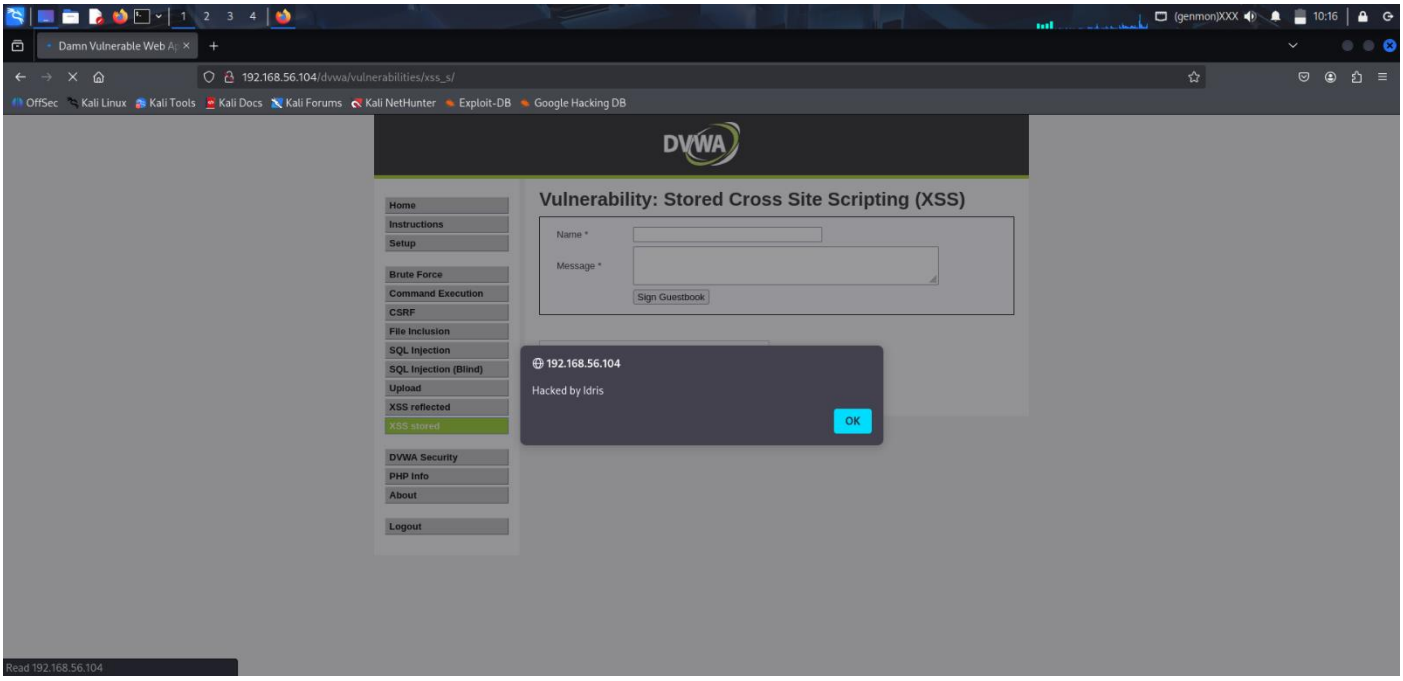
The application "reflects" user input directly back to the browser without proper encoding, allowing for the execution of malicious JavaScript.

## Exploitation Steps

1. Navigated to the XSS (Reflected) module.
2. Injected the payload: <script>alert('XSS by Idris')</script>
3. Upon clicking submit, the browser executed the script, triggering an alert box.

## Risk Impact

High. This can be used to steal session cookies (document.cookie), redirect users to malicious sites, or perform actions on behalf of the user.




## Remediation (The Fix)

Implement Output Encoding using functions like htmlspecialchars() to convert special characters into HTML entities. Additionally, a strong Content Security Policy (CSP) should be enforced.

---

## 4. Cross-Site Request Forgery (A01:2021 – Broken Access Control)

## Vulnerability Description

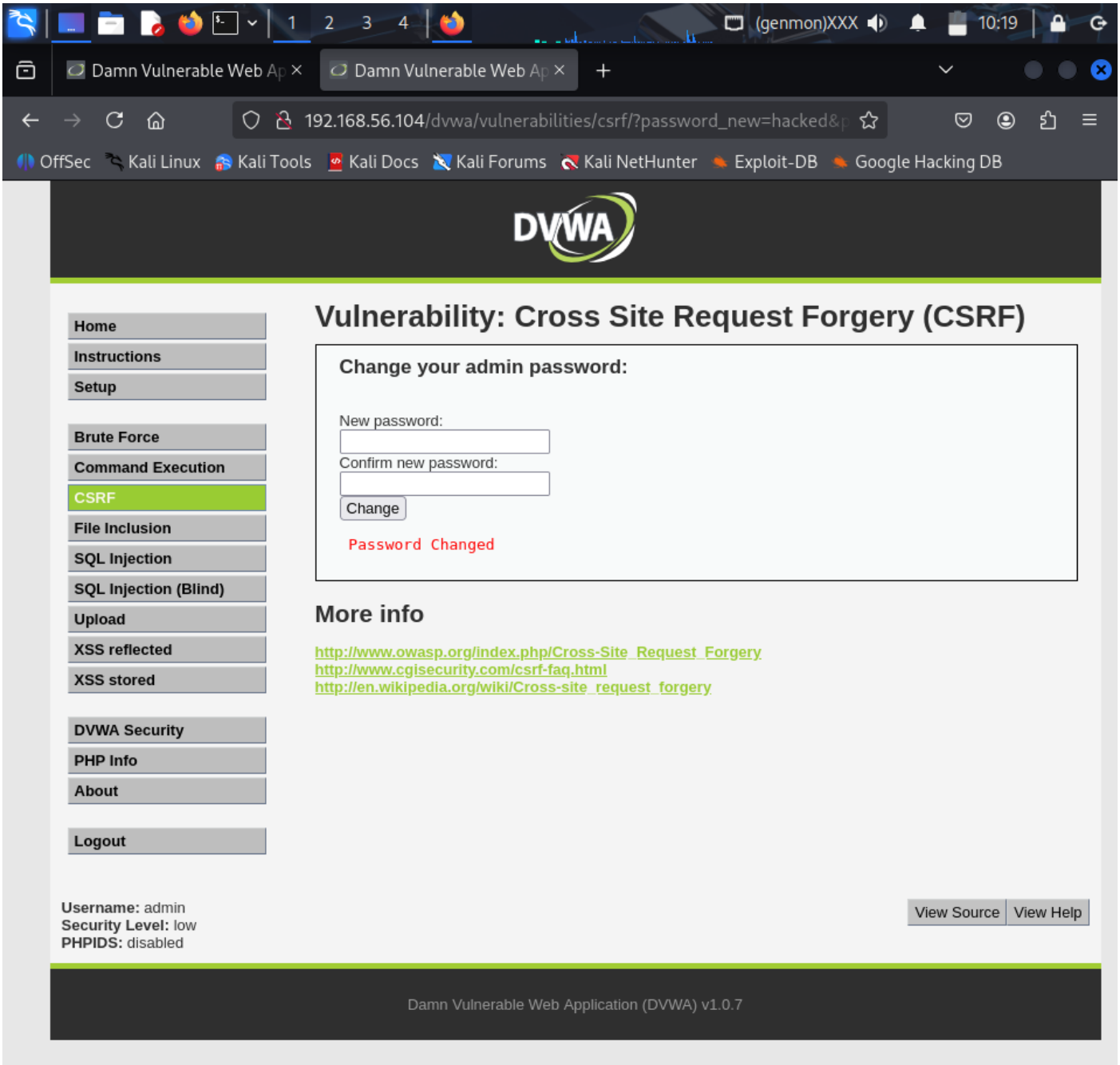
The application fails to verify if a sensitive request (changing a password) was intentionally initiated by the user. An attacker can trick a logged-in user into executing an unwanted action by hosting a malicious link or hidden form on a different site.

## Exploitation Steps

1. Navigated to the CSRF module in DVWA.
2. Observed the URL structure used to change a password: password_new=hacked&password_conf=hacked&Change=Change
3. Constructed a malicious URL (or a hidden HTML form) that, when clicked by the victim while they are logged into DVWA, automatically changes their password to "hacked" without their consent.

## Risk Impact

High. An attacker can perform administrative actions, change user credentials, or delete data by exploiting the existing session of an authenticated user.




## Remediation (The Fix)

The most effective defense is the implementation of Anti-CSRF Tokens. These are unique, cryptographically strong, and unpredictable strings generated by the server for every user session or request.

---

## 5. Broken Access Control (Proxy Manipulation)

## Vulnerability Description

The application relies on client-side data that can be intercepted and modified before reaching the server.

### Exploitation Steps

1. Intercepted a login request using Burp Suite Professional.
2. Captured a request with an incorrect password: password=wrongpassword.
3. Manually modified the parameter in the Burp Proxy to: password=password.
4. Forwarded the modified request, successfully gaining access to the administrative dashboard.

### Risk Impact

Medium/High. Demonstrates that client-side security checks can be bypassed if the server does not perform robust re-validation of all parameters.

---

## 6. Security Headers Analysis

### Findings

An analysis of the target's HTTP response headers revealed several missing security configurations:

- Missing: Content-Security-Policy (Prevents XSS)
- Missing: X-Frame-Options (Prevents Clickjacking)
- Missing: Strict-Transport-Security (Forces HTTPS)

### Remediation

Update the Apache configuration (.htaccess or httpd.conf) to include these headers:

### Apache

Header set X-Frame-Options "DENY"

Header set Content-Security-Policy "default-src 'self';"

---

## 7. Conclusion

The web application is currently vulnerable to multiple high-impact attacks. By implementing Input Validation, Output Encoding, and Prepared Statements, the organization can significantly harden the infrastructure against the most common web-based threats.